



دانشکده علوم ریاضی
دانشگاه فردوسی مشهد

پروژه دوم درس طراحی الگوریتم

احداث شبکه درختی پایه با درخت پوشای کمینه

استاد درس: دکتر رضا قنبری

اعضای گروه:

سارا مسگرمقدم ، اسما موحدی تبار

فهرست مطالب

۱	تعریف مسئله	۲
۲	چرا الگوریتم کروسکال؟	۲
۱.۲	مقایسه‌ی کروسکال و پریم	۲
۲.۲	دلیل انتخاب کروسکال برای این پروژه	۲
۳	ساختار داده‌ی Union-Find	۳
۱.۳	عملیات find با فشردگی مسیر (Compression Path)	۳
۲.۳	عملیات union با اتحاد بر اساس رتبه Rank (by Union)	۳
۴	شرح الگوریتم کروسکال گام به گام	۴

۱ تعریف مسئله

در ابتدای بازسازی شبکه‌ی مخابراتی امپراتوری، دپارتمان ارتباطات فهرستی از n شهر و e کابل پیشنهادی (همراه با هزینه‌ی ساخت هر کابل) در اختیار ما قرار می‌دهد. هدف این فاز، طراحی شبکه‌ی اصلی به گونه‌ای است که تمام شهرها با کمترین هزینه‌ی ممکن به یکدیگر متصل شوند. از نظر نظریه‌ی گراف، این مسئله دقیقاً معادل یافتن درخت پوشای کمینه (MST) یک گراف وزن‌دار و بدون جهت است: زیرمجموعه‌ای از یال‌ها که (۱) تمام n راس را همبند کند، (۲) هیچ دوری (Cycle) نداشته باشد و در نتیجه دقیقاً $n - 1$ یال داشته باشد، و (۳) مجموع وزن یال‌های انتخابی کمینه باشد.

۲ چرا الگوریتم کروسکال؟

برای حل مسئله‌ی MST دو الگوریتم حریصانه‌ی کلاسیک و اثبات‌شده وجود دارد: کروسکال (Kruskal) درخت پوشای کمینه تفاوت اصلی آن‌ها در نحوه‌ی ساخت درخت و ساختار داده‌ی مورد نیازشان است.

۱.۲ مقایسه‌ی کروسکال و پریم

- پریم از یک راس دلخواه شروع می‌کند و در هر مرحله، ارزان‌ترین یالی را که یک سرش داخل درخت در حال ساخت و سر دیگرش خارج از آن است اضافه می‌کند. این رویکرد برای گراف‌های چگال (که تعداد یال‌هایشان نزدیک به n^2 است) و با استفاده از ساختار داده‌ی صف اولویت (Queue Priority) کارآمدتر است.
- کروسکال تمام یال‌های گراف را یک‌بار بر اساس وزن مرتب می‌کند، سپس یال‌ها را به ترتیب صعودی وزن بررسی می‌کند و هر یالی را که باعث تشکیل دور نشود اضافه می‌کند. این رویکرد برای گراف‌های کم چگال (که تعداد یال‌هایشان بسیار کمتر از n^2 است) کارآمدتر است، زیرا صرفاً به یک مرتب‌سازی اولیه و یک ساختار داده‌ی ساده برای تشخیص دور نیاز دارد.

۲.۲ دلیل انتخاب کروسکال برای این پروژه

در این پروژه، ورودی به صورت لیست یال‌ها (نه ماتریس مجاورت) داده می‌شود و طبیعت مسئله (شبکه‌ی ارتباطی بین شهرها) معمولاً منجر به گراف‌های نسبتاً کم چگال می‌شود (تعداد کابل‌های پیشنهادی معمولاً بسیار کمتر از n^2 است، چون کشیدن کابل بین هر دو شهر دلخواه از نظر اقتصادی منطقی نیست). کروسکال از نظر پیاده‌سازی نیز ساده‌تر است، زیرا نیازی به ساخت لیست مجاورت یا صف اولویت پیچیده ندارد و صرفاً به یک مرتب‌سازی اولیه و

ساختار داده‌ی مجموعه‌های مجزا (Union-Find) برای تشخیص دور نیاز دارد. به همین دلایل، کروسکال برای این فاز از پروژه انتخاب شد.

۳ ساختار داده‌ی Union-Find

قلب پیاده‌سازی کروسکال، ساختار داده‌ی Union-Find است که پاسخ سریع به این پرسش را ممکن می‌سازد: «آیا دوراس u و v از قبل در یک مؤلفه‌ی همبند قرار دارند؟» این سؤال برای تشخیص اینکه آیا افزودن یک یال باعث تشکیل دور می‌شود یا نه، حیاتی است.

۱.۳ عملیات find با فشردگی مسیر (Compression Path)

```

۱
۲ static int find(int x) {
۳     if (parent[x] != x) {
۴         parent[x] = find(parent[x]);
۵     }
۶     return parent[x];
۷ }
```

این تابع به صورت بازگشتی، ریشه‌ی مؤلفه‌ای که راس x در آن قرار دارد را می‌یابد. خط `parent[x] = find(parent[x])` پس از یافتن ریشه، مسیر طی شده را فشرده می‌کند: تمام راس‌های واقع در این مسیر، مستقیماً به ریشه متصل می‌شوند. این تکنیک باعث می‌شود فراخوانی‌های بعدی `find` برای همان راس‌ها تقریباً در زمان ثابت انجام شوند.

۲.۳ عملیات union با اتحاد بر اساس رتبه Rank (by Union)

```

۱ static boolean union(int x, int y){
۲     int px = find(x);
۳     int py = find(y);
۴     if (px == py){
۵         return false;
۶     }
```

```

۷     if (parentRank[px] < parentRank[py]) {
۸         int t = px; px = py; py = t;
۹     }
۱0    parent[py] = px;
۱۱    if (parentRank[px] == parentRank[py]){
۱۲        parentRank[px]++;
۱۳    }
۱۴    return true;
۱۵ }

```

اگر ریشه‌های دو راس یکسان باشند ($px == py$)، این دو راس از قبل در یک مؤلفه قرار دارند و اتحادشان باعث تشکیل دور می‌شود؛ در این صورت تابع مقدار `false` برمی‌گرداند. در غیر این صورت، ریشه‌ای با رتبه‌ی کوچک‌تر همواره زیر ریشه‌ای با رتبه‌ی بزرگ‌تر قرار می‌گیرد. این استراتژی، که اتحاد بر اساس رتبه نامیده می‌شود، تضمین می‌کند ارتفاع درخت‌های Union-Find همواره از $O(\log n)$ تجاوز نکند، زیرا ارتفاع تنها زمانی افزایش می‌یابد که دو درخت هم‌رتبه با یکدیگر ادغام شوند (که در این حالت، رتبه‌ی ریشه‌ی جدید یک واحد افزایش می‌یابد).

۴ شرح الگوریتم کروسکال گام‌به‌گام

۱. مرتب‌سازی یال‌ها: تمام e یال ورودی بر اساس هزینه‌ی ساخت، به‌صورت صعودی مرتب می‌شوند.
۲. مقداردهی اولیه‌ی Union-Find: هر راس ابتدا به‌عنوان یک مؤلفه‌ی مستقل و تک‌عضوی در نظر گرفته می‌شود ($\text{parent}[i] = i$).
۳. پیمایش یال‌های مرتب‌شده: به ترتیب از ارزان‌ترین یال شروع می‌شود. برای هر یال (u, v, w) ، عملیات $\text{union}(u, v)$ فراخوانی می‌شود.
 - اگر union مقدار `true` برگرداند (یعنی u و v در دو مؤلفه‌ی متفاوت بودند)، این یال به MST اضافه شده و هزینه‌اش به مجموع افزوده می‌شود.
 - اگر union مقدار `false` برگرداند (یعنی u و v از قبل در یک مؤلفه بودند)، این یال باعث تشکیل دور می‌شد و نادیده گرفته می‌شود.
۴. توقف زودهنگام: به محض آنکه تعداد یال‌های انتخاب‌شده به $n - 1$ برسد، پیمایش متوقف می‌شود، زیرا یک درخت پوشا با n راس دقیقاً $n - 1$ یال دارد و بررسی یال‌های باقی‌مانده دیگر ضروری نیست.